

Guia 4 — Branches, Navegação e Fluxo Profissional

Branches são um dos conceitos mais importantes — e mais poderosos — do Git.

Elas permitem criar **linhas paralelas de desenvolvimento**, testar ideias, corrigir bugs e desenvolver novas funcionalidades **sem tocar no código principal**.

Em outras palavras:

Branches são como universos paralelos do seu projeto.

Você cria, modifica, experimenta... e só junta ao código principal quando estiver pronto.

Neste guia, você vai aprender a trabalhar com branches de forma profissional: criar, navegar, mesclar, resolver conflitos e entender fluxos reais usados por equipes de desenvolvimento.

1. O que é uma branch (de verdade)

Uma **branch** é uma linha do tempo independente dentro do seu projeto.

Ela permite que você desenvolva algo novo **sem mexer no código principal**.

✓ Analogia simples (e poderosa)

Imagine que o projeto é uma estrada:

- **main** → a estrada principal, onde tudo precisa estar funcionando
- **feature/login** → uma rua lateral onde você trabalha na nova funcionalidade
- quando termina, você volta para a estrada principal (merge)

Essa metáfora explica tudo o que importa:

- você cria um caminho separado
- trabalha sem atrapalhar ninguém
- depois junta tudo de volta

✓ Por que branches existem?

Branches permitem:

- desenvolver novas funcionalidades sem quebrar o projeto
- corrigir bugs isoladamente
- testar ideias sem risco
- trabalhar em equipe sem conflitos constantes
- manter o código principal sempre estável

✓ O que você ganha usando branches?

- organização
- segurança
- histórico limpo

- fluxo profissional
- facilidade para colaborar

Branches são a base de qualquer fluxo moderno de Git — desde projetos solo até equipes grandes.

✂ 2. Ver branches existentes — `git branch`

O comando abaixo mostra **todas as branches locais** do seu repositório:

```
git branch
```

✓ Exemplo real de saída

```
* main  
feature-login  
ajustes-css
```

✓ Como interpretar

- * main → a branch atual (onde você está trabalhando agora)
- feature-login → branch local existente
- ajustes-css → outra branch local

Importante:

- git branch mostra apenas branches locais.
- Branches remotas (do GitHub) aparecem com:
 - git branch -r → apenas branches remotas
 - git branch -a → branches locais + remotas
 - git fetch → atualiza a lista de branches remotas

Saber visualizar suas branches é essencial para navegar com segurança e entender onde você está trabalhando dentro do projeto.

3. Criar uma branch — `git branch nome`

Para criar uma nova branch **sem trocar para ela**, use:

```
git branch feature-login
```

Se tudo der certo, nenhuma saída aparece — esse é o comportamento normal do Git.

✓ Verificando se a branch foi criada

```
git branch
```

Saída típica:

```
feature-login  
* main
```

✓ Como interpretar a saída

- feature-login → a branch foi criada com sucesso
- * main → você ainda está na branch main
- criar uma branch não muda a branch atual

💡 Dica importante

Criar uma branch é como abrir uma nova rua, mas você ainda está parado na rua antiga.

Para entrar na nova rua, você precisa usar git switch (na próxima seção).

4. Trocar de branch — `git switch` (recomendado)

Depois de criar uma branch, você precisa **entrar nela** para começar a trabalhar.

O comando recomendado para isso é:

```
git switch feature-login
```

Saída típica:

```
Switched to branch 'feature-login'
```

✓ O que aconteceu?

- agora você está dentro da branch feature-login
- qualquer commit que fizer será registrado nessa branch
- sua pasta de trabalho muda para refletir o conteúdo dessa branch

🔗 Alternativa antiga (ainda funciona): git checkout

Antes do Git 2.23, o comando usado para trocar de branch era:

```
git checkout feature-login
```

Ele ainda funciona, mas hoje é menos recomendado, porque:

- checkout faz muitas coisas diferentes (trocar branch, restaurar arquivos, criar branch...)
- switch é mais claro e mais seguro para navegação entre branches

Regra prática

- Use git switch para trocar de branch.
- Use git checkout apenas quando realmente precisar (ex.: restaurar arquivos).

5. Criar e trocar ao mesmo tempo — `git switch -c`

Se você quer **criar uma nova branch** e **entrar nela imediatamente**, use:

```
git switch -c feature-login
```

Saída típica

```
Switched to a new branch 'feature-login'
```

✓ O que aconteceu aqui?

- a branch feature-login foi criada
- você já entrou nela automaticamente
- sua pasta de trabalho agora reflete o conteúdo dessa nova branch

✓ Quando usar esse comando?

- ao iniciar uma nova funcionalidade
- ao começar um ajuste isolado
- ao criar branches de correção (hotfix/*)
- sempre que quiser agilizar o fluxo sem rodar dois comandos (git branch + git switch)

Regra prática

Se você vai criar uma branch e trabalhar nela imediatamente, use git switch -c.

6. Ver em qual branch estou — `git status`

A forma mais simples e direta de descobrir **em qual branch você está trabalhando** é usando:

```
git status
```

✓ Exemplo real de saída no terminal

```
On branch feature-login
```

✓ O que isso significa?

- você está atualmente na branch feature-login
- qualquer commit que fizer será registrado nessa branch
- sua pasta de trabalho reflete o conteúdo dessa branch

💡 Dica prática

Sempre que estiver em dúvida sobre “onde você está”, rode git status.

Ele mostra a branch atual e o estado dos arquivos — tudo em um só comando.

7. Mesclar branches — git merge

Mesclar (*merge*) significa **juntar o trabalho de uma branch dentro de outra**.

O fluxo profissional é sempre o mesmo:

1. terminar o trabalho na branch secundária
2. voltar para a branch principal
3. mesclar a branch secundária nela

✓ Exemplo real

```
git switch main  
git merge feature-login
```

Saída típica

```
Updating 1a2b3c4..9f8e7d6  
Fast-forward  
arquivo.py | 10 ++++++++
```

✓ O que significa essa saída?

- Updating 1a2b3c4..9f8e7d6 → o Git avançou o ponteiro da branch

- Fast-forward → não houve divergência; o Git apenas “andou para frente”
- arquivo.py | 10 ++++++++ → 10 linhas foram adicionadas

Esse é o merge mais simples e mais comum.

⚠ Quando há conflito

Se as duas branches modificaram a mesma parte do mesmo arquivo, o Git não sabe qual versão escolher:

```
CONFLICT (content): Merge conflict in app.py
```

O arquivo ficará assim:

```
<<<<<< HEAD
código da branch atual
=====
código da branch que está sendo mesclada
>>>>>> feature-login
```

✓ Como resolver

- Abra o arquivo em conflito.
 - Analise os dois blocos:
 - HEAD → código da branch atual
 - feature-login → código da branch que está sendo mesclada
- Escolha o que deve permanecer:
 - só o código de cima
 - só o de baixo
 - ou uma combinação dos dois
- Remova as marcações (<<<<<<, =====, >>>>>>).
- Salve o arquivo.
- Finalize:

```
git add app.py
git commit
```

✓ O commit finaliza o merge

Esse commit não é um commit comum — ele representa a resolução do conflito.

! Dica prática

Conflitos são normais. Eles não significam erro — apenas que duas pessoas mexeram no mesmo trecho de código.

🔗 Exemplo simples de resolução

Arquivo com conflito:

```
<<<<<< HEAD
print("Olá")
=====
print("Hello")
>>>>>> feature-login
```

Se você quer manter apenas "Olá", deixe assim:

```
print("Olá")
```

Pronto — conflito resolvido manualmente.

🗑️ 8. Deletar branches — `git branch -d`

Depois que uma branch já foi **mesclada** (merge concluído), você pode removê-la com segurança:

```
git branch -d feature-login
```

Saída típica:

```
Deleted branch feature-login (was 9f8e7d6).
```

✓ O que isso significa na prática?

- a branch feature-login foi removida do seu repositório local
- o último commit dela era 9f8e7d6
- o Git só permite deletar porque ela já foi mesclada

⚠️ Quando a branch NÃO foi mesclada

Se você tentar deletar uma branch que ainda não foi integrada, o Git protege você:

```
error: The branch 'feature-login' is not fully merged.
```

Isso evita perda accidental de trabalho.

🔥 Forçar a remoção — `git branch -D`

Se você tem certeza absoluta de que quer apagar a branch mesmo sem merge:

```
git branch -D feature-login
```

✓ Quando usar o -D?

- branch criada por engano
- testes temporários
- trabalho descartado
- branch duplicada
- branch que não será mais usada

Regra prática:

- Use -d sempre que possível.
- Use -D apenas quando tiver certeza de que não precisa mais daquela branch.

9. Branch local vs branch remota (diferença clara e como se comunicam)

Essa é uma das partes mais importantes do Git:

uma branch local e uma branch remota são coisas diferentes, mesmo que tenham o mesmo nome.

Branch local

É a branch que existe **somente no seu computador**.

Exemplo de branches locais:

```
main  
feature-login  
ajustes-css
```

Essas branches só aparecem para você — ninguém no GitHub vê isso até você enviar.

Branch remota

É a branch que existe no servidor, normalmente no GitHub.

Exemplo:


```
origin/main  
origin/feature-login
```

O prefixo **origin/** significa:

- “essa branch está no repositório remoto chamado origin”
- origin é o nome padrão do remoto criado ao fazer git clone

Como local e remoto se comunicam

Quando você cria uma branch local:

```
git switch -c feature-login
```

Ela não existe no GitHub ainda.

Para enviar essa branch para o remoto:

```
git push -u origin feature-login
```

Saída típica:

```
new branch 'feature-login' created on remote
```

Agora você tem:

- local: **feature-login**
- remoto: **origin/feature-login**

E o **-u** faz algo essencial:

✓ Ele cria o “vínculo” entre as duas branches

Depois disso:

- it push sabe para onde enviar
- git pull sabe de onde puxar

você não precisa mais escrever **origin feature-login**



Baixar branches remotas criadas por outras pessoas

Se alguém criou uma branch no GitHub, atualize sua lista de branches remotas:

```
git fetch
```

Agora veja as branches remotas:

```
git branch -r
```

Exemplo:

```
origin/main  
origin/feature-login
```

Se a branch já existir localmente:

```
git switch feature-login
```

Quando a branch remota NÃO existe localmente

Crie uma branch local baseada na remota:

```
git switch -c feature-login origin/feature-login
```

Isso significa:

- crie uma branch local chamada **feature-login**
- baseada na branch remota **origin/feature-login**

Resumo visual da comunicação

```
LOCAL → git push → REMOTO  
REMOTO → git pull → LOCAL  
REMOTO → git fetch → atualiza referências
```

- **push** envia commits
- **pull** baixa commits e integra
- **fetch** só atualiza referências (não altera sua branch atual)

✓ Regra prática para nunca esquecer

- **Branch local** = só você vê
- **Branch remota** = está no GitHub
- Elas só se conectam quando você faz:

```
git push -u origin nome-da-branch
```



10. Como funciona a comunicação entre local e remoto

O Git trabalha sempre com **duas versões da mesma branch**:

- a versão **local** (no seu computador)
- a versão **remota** (no GitHub, chamada **origin/nome-da-branch**)

Os comandos abaixo controlam como essas duas versões trocam informações.



git push — envia do local → para o remoto

Quando você faz:

```
git push
```

Você está dizendo:

“Pegue meus commits locais e envie para a branch remota correspondente.”

✓ O que realmente acontece

- a branch remota é atualizada
- seus commits passam a existir no GitHub
- outras pessoas podem ver seu trabalho



git pull — baixa do remoto → para o local

```
git pull
```

Significa:

“Baixe os commits do GitHub e integre na minha branch local.”

✓ O que realmente acontece ao fazer **git pull**

- baixa commits novos
- faz merge automático (ou rebase, dependendo da configuração)
- sua branch local fica atualizada com o remoto

`git fetch` — atualiza referências remotas (sem alterar sua branch)

```
git fetch
```

Significa:

“Atualize a lista de branches e commits do servidor, mas não mexa na minha branch atual.”

✓ O que realmente acontece com o `git fetch`

- baixa informações do remoto
- atualiza `origin/main`, `origin/feature-x`, etc.
- não altera seus arquivos
- não faz merge automático

Resumo rápido

```
git push    → envia commits do local para o remoto
git pull    → baixa commits do remoto e integra no local
git fetch   → atualiza informações do remoto sem alterar nada localmente
```

Regra prática:

- Use `fetch` para ver o que mudou.
- Use `pull` para trazer as mudanças.
- Use `push` para enviar suas mudanças.

11. Fluxo profissional de trabalho com branches

Equipes profissionais usam branches para organizar o desenvolvimento e manter o código sempre estável. O modelo mais comum é baseado em **branches com funções específicas**, cada uma com um propósito claro dentro do ciclo de desenvolvimento.

Branches principais do fluxo profissional

`main`

- código estável
- representa o que está em produção
- deve estar sempre funcionando

`develop` (opcional, mas comum em equipes)

- integra várias features antes de ir para a `main`
- funciona como um “ambiente de testes” do time

feature/*

- usadas para desenvolver novas funcionalidades
- cada funcionalidade tem sua própria branch
- exemplos:
 - `feature/login`
 - `feature/dashboard`
 - `feature/export-relatorios`

hotfix/*

- correções urgentes em produção
- criadas a partir da `main`
- exemplo:
 - `hotfix/corrige-erro-de-login`

release/*

- preparação para uma nova versão
- ajustes finais antes de enviar para produção

Fluxo típico de trabalho com branches

Este é o fluxo mais comum e recomendado:

```
git switch -c feature/login    ← cria e entra na branch da nova funcionalidade
... trabalha, faz commits ...
git switch main               ← volta para a branch principal
git merge feature/login       ← integra a funcionalidade
git push                     ← envia para o GitHub
git branch -d feature/login   ← remove a branch local
```

✓ O que esse fluxo garante?

- cada funcionalidade fica isolada
- o código da `main` permanece estável
- o histórico fica limpo e organizado
- conflitos são reduzidos
- colaboração entre pessoas fica muito mais fácil

Regra prática para nunca esquecer

- `feature` → criar coisas novas
- `hotfix` → corrigir emergências
- `release` → preparar versões
- `develop` → integrar trabalho do time

- main → código estável e pronto para produção

Esse fluxo é usado em empresas, times grandes e projetos open source.

Resumo do Guia 3

Os comandos essenciais para trabalhar com branches no Git:

<code>git branch</code>	→ lista branches locais
<code>git branch nome</code>	→ cria uma nova branch local
<code>git switch nome</code>	→ troca para outra branch
<code>git switch -c nome</code>	→ cria e já troca para a nova branch
<code>git merge nome</code>	→ mescla uma branch na branch atual
<code>git branch -d nome</code>	→ deleta branch local (se já foi mesclada)
<code>git push -u origin nome</code>	→ envia branch local para o remoto e cria o vínculo
<code>git fetch</code>	→ atualiza informações das branches remotas
<code>git pull</code>	→ baixa e integra mudanças do remoto

Fim do Guia 3 — Branches e Navegação

Agora você entende:

- o que é uma branch e por que ela existe
- como criar, trocar, mesclar e deletar branches
- a diferença entre branch local e branch remota
- como elas se comunicam (push, pull, fetch)
- como trabalhar com branches de forma profissional
- como organizar seu fluxo de desenvolvimento

Com isso, você fecha o ciclo essencial do Git:

```
status → staging → commits → branches → merges → remoto
```

Esse conhecimento é a base para trabalhar com Git de forma segura, organizada e profissional.
